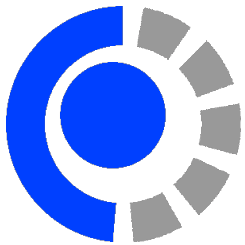
Excepciones

Programación Concurrente

Lucas A. Margni





¿Qué es una excepción?

Idea

Qué es una excepción?

Errores que se producen en tiempo de ejecución. Simplemente eso

- División por cero.
- Raíz cuadrada de un número negativo.
- Acceder a una posición inexistente de un arreglo.
- Ejecutar un método de un objeto que es null.
- Leer un archivo que no existe.
- ...

Qué sucede cuando se produce una excepción?

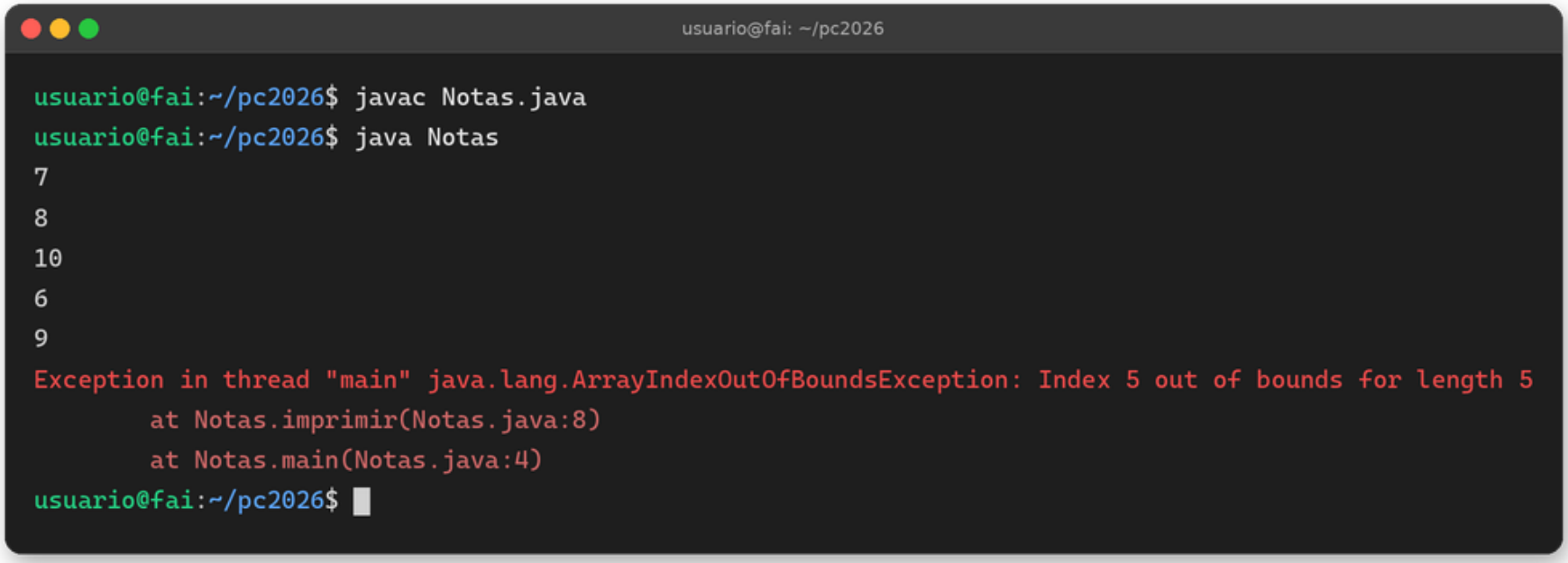
Se lanza un objeto denominado **Throwable**, el cual puede ser capturado

Lo vieron mil veces

```
public class Notas {
  public static void main(String[] args) {
    int[] notas = {7, 8, 10, 6, 9};
    imprimir(notas);
  }
  static void imprimir(int[] v) {
    for (int i = 0; i <= v.length; i++) {
      System.out.println(v[i]);
    }
  }
}
```

Compila sin una sola advertencia. ¿Qué pasa al ejecutarlo?

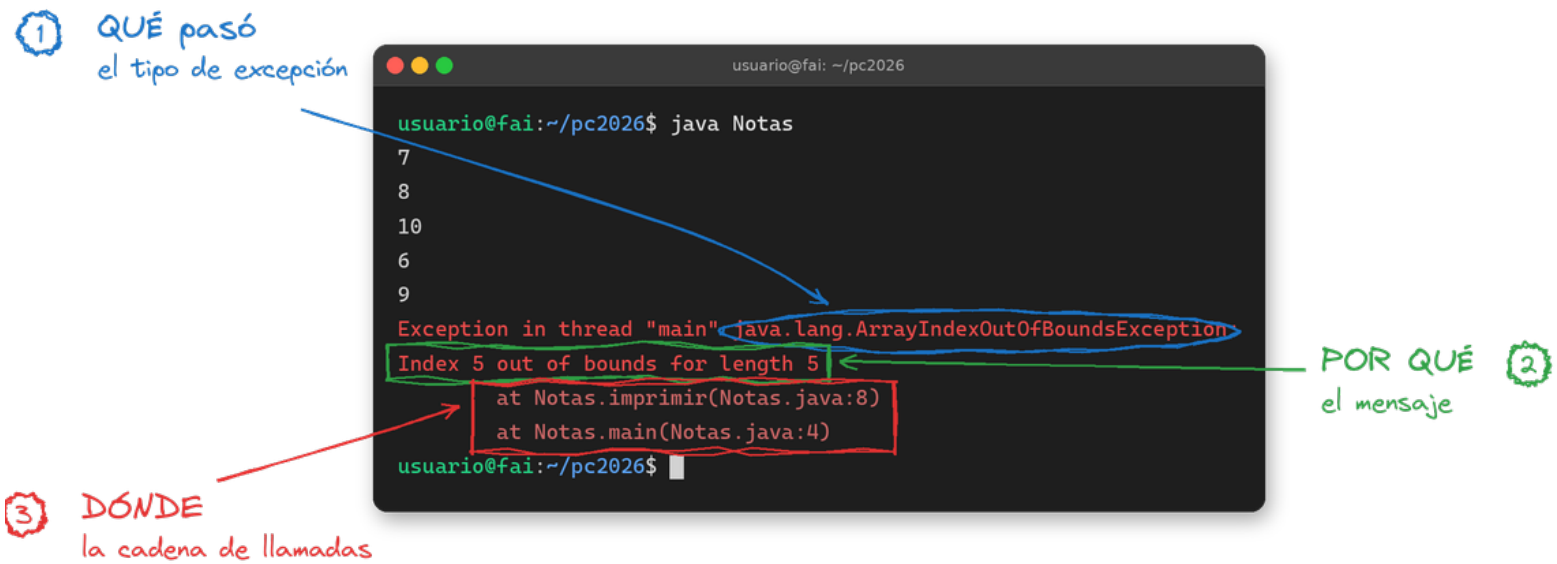
Lo vieron mil veces



No da un resultado incorrecto: el programa muere.

Imprimió las 5 notas y explotó al pedir la sexta.

Anatomía del error



se lee de arriba hacia abajo: primero dónde explotó, después quién lo llamó

Tres datos, siempre

- **Qué** pasó (el tipo)
- **Por qué** (el mensaje)
- **Dónde** (la pila de llamadas)

El problema

Un error en tiempo de ejecución rompe el programa...

...pero muchas veces **sí sabemos qué hacer** si ocurre:

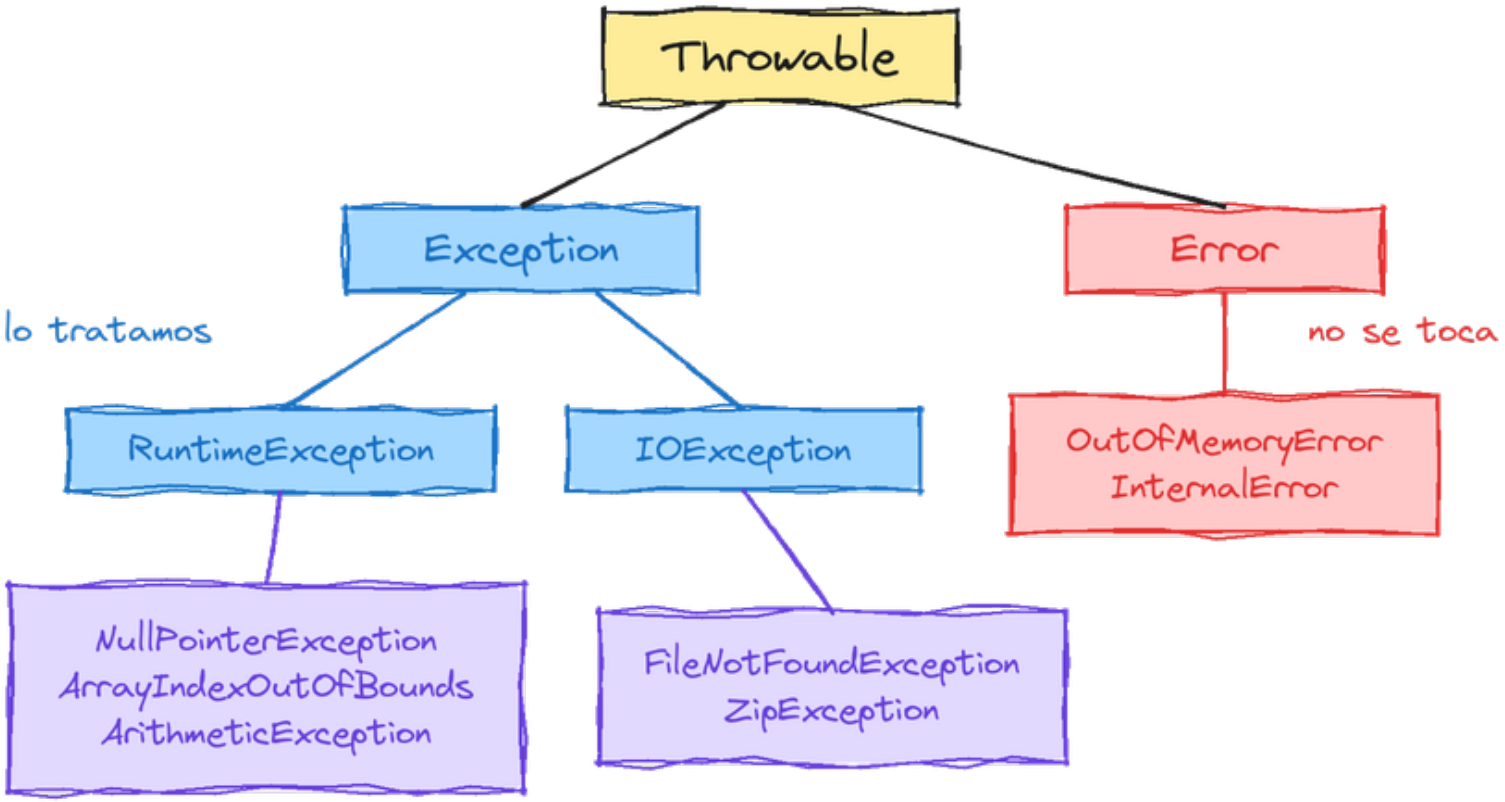
- ¿El usuario escribió mal? Volver a preguntar.
- ¿No existe el archivo? Usar valores por defecto.
- ¿Se cayó la conexión? Reintentar.

Necesitamos identificar tres cosas

- El **bloque protegido**: el código que queremos vigilar.
- El **tipo de excepción** que nos interesa.
- El **manejador**: qué hacer cuando ocurre.

La jerarquía de Java

Todo lo que se puede lanzar



Todo lo que se lanza en Java es un **objeto**.
Y todo objeto lanzable hereda de **Throwable**.

Throwable

La clase base

Representa todo lo que se puede "lanzar" en Java.

- Una **instantánea de la pila** en el momento en que se creó (*stack trace*).
- Un **mensaje** (String) que describe qué pasó.
- Opcionalmente una **causa**: otro Throwable, el error que provocó este error.

Es un objeto común y corriente: tiene estado y métodos.
El más usado: `getMessage()`.

Error vs. Exception

Error

Problemas graves que la aplicación **no debería** intentar solucionar.

`OutOfMemoryError`
`StackOverflowError`
`InternalError`

No se capturan.

Exception

Situaciones que la aplicación **debería** tratar de forma razonable.

`NullPointerException`
`FileNotFoundException`
`ArithmeticException`

Acá trabajamos nosotros.

Las dos partes de Exception

RuntimeException

Errores **del programador**: división por cero, acceso fuera de los límites de un array, usar una referencia nula.

IOException

Errores que el programador **no puede evitar**: el archivo no está, la red se cayó, el disco está lleno.

La distinción importa: al compilador **no** le preocupan las primeras, pero **sí** las segundas.

Las Runtime se arreglan programando mejor.
Las IO hay que contemplarlas.

Capturar: try / catch

La estructura

```
// Bloque 1
try {
    // Bloque 2 ← bloque protegido
} catch (Exception error) {
    // Bloque 3 ← manejador
}
// Bloque 4
```

- En cuanto se produce la excepción, **el bloque try termina**. No se retoma.
- La cláusula `catch` recibe como argumento el objeto Throwable que se lanzó.
- Si el `catch` maneja la excepción, el programa **sigue vivo**.

¿Qué se ejecuta?

```
// Bloque 1
try {
    // Bloque 2
} catch (Exception error) {
    // Bloque 3
}
// Bloque 4
```

- Sin excepción: `1 - 2 - 4`
- Excepción en el Bloque 2: `1 - 2* - 3 - 4`
- Excepción en el Bloque 1: `1*`

2* significa: el bloque 2 se ejecutó **hasta** el punto donde falló.

Varios manejadores

```
// Bloque 1
try {
    // Bloque 2
} catch (ArithmeticException ae) {
    // Bloque 3
} catch (NullPointerException ne) {
    // Bloque 4
}
// Bloque 5
```

- Sin excepción: `1 - 2 - 5`
- Excepción aritmética: `1 - 2* - 3 - 5`
- Acceso a un objeto nulo: `1 - 2* - 4 - 5`
- **Excepción de otro tipo:** `1 - 2*`

Si ningún catch coincide, la excepción **sigue viva** y el programa muere igual.

Una red de seguridad

```
// Bloque 1
try {
    // Bloque 2
} catch (ArithmeticException ae) {
    // Bloque 3
} catch (Exception error) {
    // Bloque 4 ← atrapa todo lo demás
}
// Bloque 5
```

- Sin excepción: `1 - 2 - 5`
- Excepción aritmética: `1 - 2* - 3 - 5`
- Cualquier otra excepción: `1 - 2* - 4 - 5`

Ahora el programa **nunca** muere en ese bloque.

¿Y acá qué pasa?

```
// Bloque 1
try {
    // Bloque 2
} catch (Exception error) {
    // Bloque 3
} catch (ArithmeticException ae) {
    // Bloque 4
}
// Bloque 5
```

¿Cuándo se ejecuta el Bloque 4?

Nunca. Y el compilador lo sabe

Error de compilación

```
exception ArithmeticException has already been caught
```

La regla

Las cláusulas se comprueban **en orden**. Los tipos más **específicos** van antes que los más **generales**.

Como `ArithmeticException` **es una** `Exception`, el primer catch ya la atrapa. El segundo es código inalcanzable.

Regla práctica: de las hojas hacia la raíz del árbol.

La cláusula finally

Pase lo que pase

A veces hay que ejecutar código **sí o sí**: cerrar un archivo, liberar un recurso, cerrar una conexión.

```
// Bloque 1
try {
    // Bloque 2
} catch (ArithmeticException ae) {
    // Bloque 3
} finally {
    // Bloque 4
}
// Bloque 5
```

- Sin excepción: `1 - 2 - 4 - 5`
- Excepción aritmética: `1 - 2* - 3 - 4 - 5`
- Excepción de otro tipo: `1 - 2* - 4`

El detalle fino

Si el cuerpo del try empieza a ejecutarse, finally se ejecuta

- Después del `try`, si no hubo excepciones.
- Después del `catch`, si este capturó la excepción.
- Justo después de la excepción **si ningún catch la captura**, y **antes** de que se propague hacia afuera.

Miren el último caso: `1 - 2* - 4`.

El Bloque 5 no se ejecuta, pero el 4 sí.

Lanzar y propagar: throw / throws

La sentencia throw

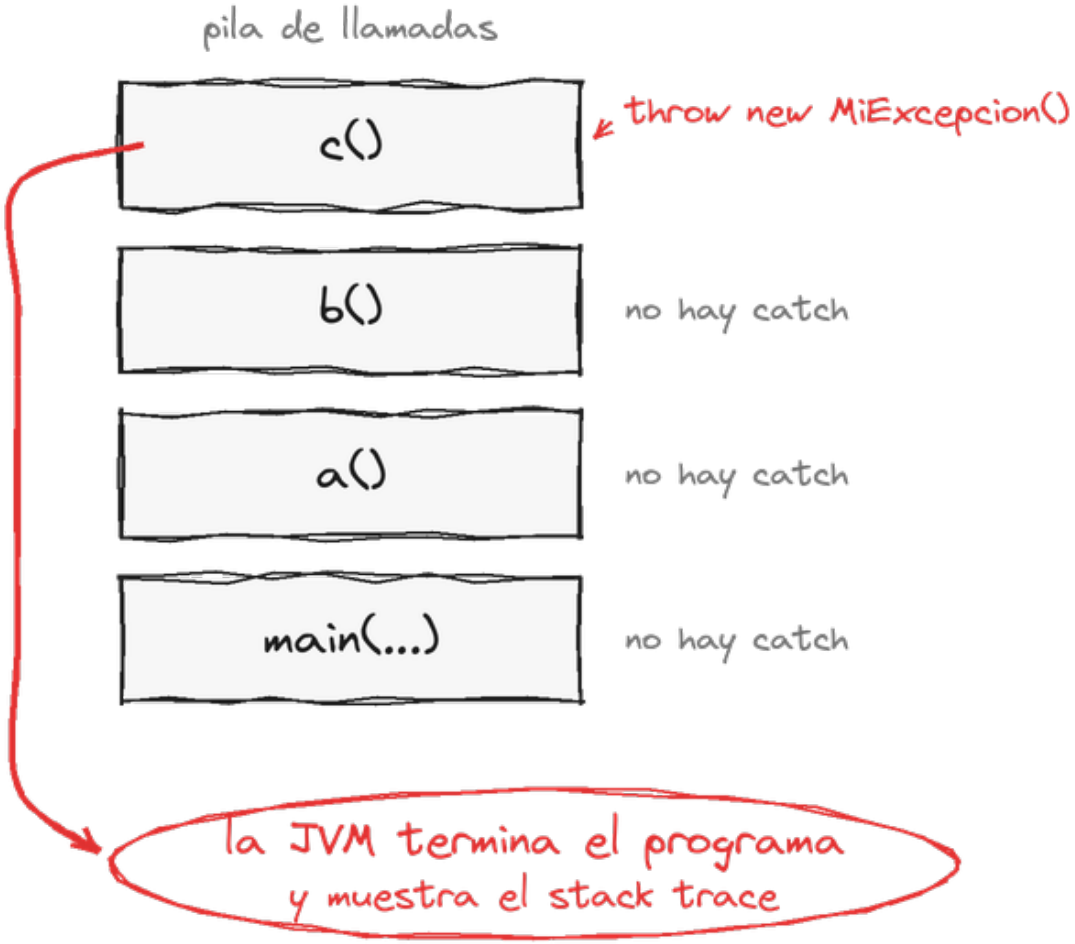
Hasta ahora **capturamos** excepciones que lanzaba Java. También podemos **lanzarlas nosotros**:

```
throw new Exception("Mensaje de error...");
```

Cuando se lanza una excepción

- Se sale **inmediatamente** del bloque actual.
- Si hay un `catch` adecuado, se ejecuta su cuerpo.
- Si no, se sale del método y se busca un catch apropiado **en quien lo llamó**.
- El proceso continúa hasta el `main`. Si ahí tampoco hay catch, la JVM termina con un mensaje de error.

Si nadie la captura...



La excepción se va **desapilando** hasta encontrar un `catch` que le sirva. Eso es el **stack trace** que vimos al principio.

Propagación: la cláusula throws

Si un método puede lanzar una excepción y **no la captura**, tiene que avisarlo en su firma:

```
public String leerArchivo(String nombreArchivo)
                              throws IOException {
    // ... codigo que puede fallar
}
```

Es un contrato

`throws` le dice a quien me llame:
"esto puede fallar, arreglátelas vos".

Las `RuntimeException` (que son muy comunes) **no** hace falta declararlas.

Dos caminos para el mismo método

1. Propagar

```
public void f() throws IOException {
    // codigo que puede lanzar una IOException
}
```

2. Capturar

```
public void f() {
    try {
        // codigo que puede lanzar una IOException
    } catch (IOException error) {
        // tratamiento de la excepcion
    } finally {
        // liberar recursos
    }
}
```

¿Cuál elijo? **¿Tengo la información para resolverlo acá?**

Excepciones propias

Crear un tipo nuevo

Basta con **heredar** de un tipo de excepción ya existente:

```
public class DivideByZeroException
                    extends ArithmeticException {
    public DivideByZeroException(String message) {
        super(message);
    }
}
```

Y se lanza como cualquier otra:

```
public double dividir(int num, int den)
                throws DivideByZeroException {
    if (den == 0)
        throw new DivideByZeroException("Error!");
    return ((double) num / (double) den);
}
```

¿Para qué molestarse?

La mayoría de las subclases de `Exception` **no agregan nada**: heredan y listo.

Entonces, ¿dónde está la información?

En tres lugares

- El **nombre** de la excepción.
- Su **lugar en la jerarquía**.
- El **mensaje** que lleva adentro.

Por eso las aplicaciones definen sus propias excepciones: para nombrar las situaciones excepcionales **de su dominio**.

Cierre

Las que se van a cruzar

Excepción	**Cuándo**
`NullPointerException`	Método sobre una referencia null
`ArrayIndexOutOfBounds...`	Índice fuera del arreglo
`ArithmeticException`	División entera por cero
`NumberFormatException`	`Integer.parseInt("hola")`
`InputMismatchException`	`nextInt()` y llega texto
`FileNotFoundException`	El archivo no existe
`ClassCastException`	Casteo inválido entre tipos

Las primeras cinco son **RuntimeException**: el compilador no las va a mencionar. Aparecen en ejecución.

Ejercicio

```
public static void main(String[] args) {
    int[] v = {1, 2, 3};
    try {
        System.out.println("A");
        System.out.println(v[5]);
        System.out.println("B");
    } catch (ArithmeticException e) {
        System.out.println("C");
    } finally {
        System.out.println("D");
    }
    System.out.println("E");
}
```

¿Qué imprime?

Ejercicio

```
A  D
```

- `A` se imprime.
- `v[5]` lanza `ArrayIndexOutOfBoundsException` → el try termina ahí, `B` nunca se imprime.
- El catch es de `ArithmeticException`: **no coincide**. `C` no se imprime.
- `finally` se ejecuta igual → `D`.
- La excepción se propaga, mata el `main`: `E` no se imprime.

Para llevarse

- Una excepción es un **objeto** que se lanza cuando algo falla en ejecución.
- `try/catch` define el bloque protegido y el manejador.
- Los catch se evalúan **en orden**: de específico a general.
- `finally` se ejecuta **siempre**.
- `throw` lanza, `throws` avisa.
- Si nadie la captura, sube hasta el `main` y el programa termina.

Capturar una excepción es decidir qué hacer,
no hacer de cuenta que no pasó nada.